

BioResearch Agent: A Tool-First Framework for Structured Biomedical Research Workflows

Alimujiang Tudiyusufu

Independent Researcher

`Alim_T@foxmail.com`

ORCID: 0009-0002-5180-6226

Abstract

BioResearch Agent is an open-source, reproducible workflow framework for structured biomedical research. It integrates three common analysis pipelines—literature review, biomarker discovery, and Mendelian randomization—behind a unified command-line interface, Python SDK, and external API invocation contract. The system emphasizes reproducible execution through version-pinned dependencies, fixed random seeds, and continuous integration across Python 3.9–3.12. A lightweight standardized interface provides a consistent entry point for external systems to invoke workflows without modifying core pipeline code. All workflows are validated via automated tests and produce standardized structured outputs, including statistical tables, co-occurrence graphs, and formatted figures. The framework is released as a frozen snapshot with archived commits and is intended as a reusable infrastructure layer rather than a replacement for domain-specific analysis tools.

1 Introduction

Biomedical research increasingly relies on fragmented computational tools spanning literature mining [1], omics analysis [2, 3], and statistical genetics [4, 5]. While large language models can assist in tool use and reasoning, they typically lack reproducible, structured execution pipelines with consistent output contracts.

We present BioResearch Agent, a tool-first framework that prioritizes reproducible execution and standardized interfaces. The system is positioned

as a reusable workflow infrastructure: given controlled inputs, it yields consistent structured outputs across runs and environments.

The framework integrates three core biomedical analysis workflows—literature review, biomarker discovery, and causal inference—behind a single reproducible workflow runner, exposed through a command-line interface, a Python software development kit, and an API invocation contract for external systems.

This work does not introduce new statistical or machine learning methods. Instead, it focuses on system-level reproducibility, interface standardization, and execution of existing biomedical analysis tools. All computational components are delegated to established external libraries such as `limma` [2], GSEA [3], and the MR-Base platform [4].

We emphasize that the primary contribution of this work is not algorithmic novelty, but the design of a reproducible execution contract that standardizes how existing biomedical analysis tools are invoked, composed, and validated across heterogeneous environments.

2 Related Work

Reproducible workflow execution in bioinformatics has been extensively addressed by rule-based systems such as Snakemake and Galaxy, which manage dependency graphs and computational environments but typically require domain-specific configuration for tool integration. Nextflow and Cromwell extend this paradigm with container-native execution and scalable cloud deployment.

More recently, large language model based scientific agent frameworks have explored language-guided automation for research tasks. However, these systems typically prioritize generative flexibility over execution stability and often lack standardized output contracts.

BioResearch Agent occupies a complementary position: rather than introducing a new workflow runner or autonomous reasoning layer, it provides a lightweight standardized interface over existing bioinformatics tools. Its primary design goal is to reduce integration friction by enforcing a unified invocation schema and consistent output structure, while delegating computation to mature domain-specific libraries such as `limma` [2], GSEA [3], and the MR-Base platform [4]. This positions the system as interoperability infrastructure rather than a replacement for existing workflow systems or agent frameworks.

3 System Overview

The system consists of four access layers over a single reproducible workflow runner:

- CLI interface
- Python SDK
- API invocation contract
- Standardized external interface (Section 4)

The workflow runner executes three analysis workflows through six processing modules. The end-to-end research process is organized as a twelve-stage execution diagram (question formulation, hypothesis generation, literature retrieval, study design, data ingestion, statistical analysis, bioinformatics, output aggregation, writing, review, critique, and formatted output). Stages 1–8 are fully implemented in the current release; stages 9–12 (writing, review, critique, formatted output) describe the intended end-to-end research workflow and are not automated by the current system.

3.1 Literature Review

PubMed-based retrieval, entity extraction, co-occurrence graph construction, and research-gap identification. Outputs include a knowledge graph and a structured gap report.

3.2 Biomarker Discovery

Gene expression analysis using differential expression and pathway enrichment. Volcano plots and enriched-pathway tables are produced as reproducible outputs. Unless a real cohort is supplied, the pipeline operates on synthetic expression matrices.

3.3 Causal Inference

Mendelian randomization using simulated GWAS summary statistics. The pipeline performs inverse-variance-weighted (IVW) estimation with sensitivity analysis, producing scatter and funnel plots. All genetic inputs are simulated and explicitly labeled as such.

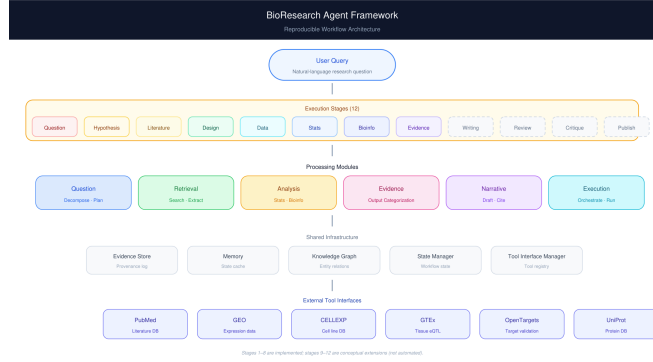


Figure 1: System component diagram of BioResearch Agent: the workflow runner, three supported analysis pipelines, and the four access layers.

4 Standardized Invocation Interface

We provide a lightweight standardized interface that exposes workflows through a unified JSON schema. This interface acts as a thin wrapper over the core workflow runner, mapping structured inputs to pipeline execution. It does not introduce new computational functionality; instead, it offers a consistent entry point for external systems to invoke existing workflows without modifying core code.

The interface is compatible with function-calling systems and provides a standardized mapping to existing pipelines. The interface package is distributed as a manifest, a schema definition, and usage instructions, and is included with this submission. No runtime extension mechanism, plugin registry, or dynamic loading is introduced; the interface package is a packaging and interface layer only.

5 System Demonstration

We demonstrate the system using three representative workflows.

Each workflow produces structured outputs including statistical tables

and formatted figures. Figure 2 shows representative outputs from the literature review, biomarker discovery, and causal inference pipelines.

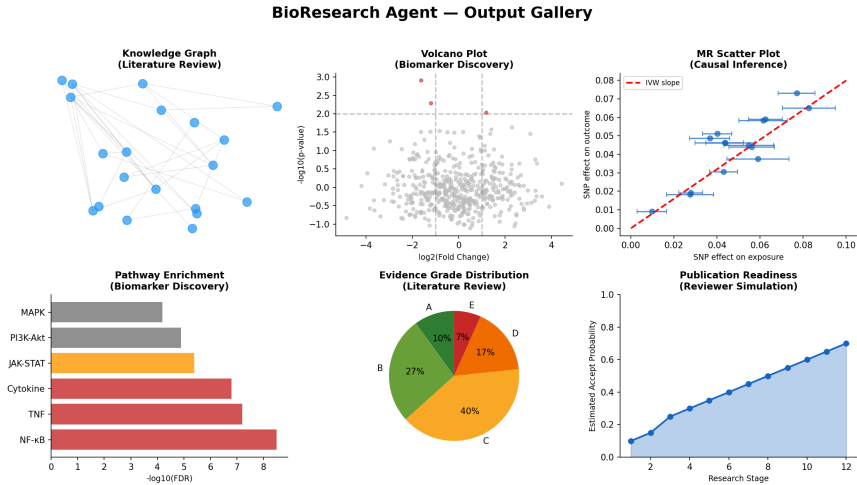


Figure 2: Representative outputs from three analysis pipelines: literature co-occurrence graph, differential-expression volcano plot, and Mendelian-randomization scatter and funnel plots.

Figure 3 shows the twelve-stage execution diagram that maps the typical biomedical research workflow from question formulation to formatted output. Stages 1–8 are implemented in the current release; stages 9–12 (writing, review, critique, formatted output) represent conceptual extensions of the end-to-end workflow and are not automated in the current system.



Figure 3: Execution diagram across 12 stages. Stages 1–8 are implemented in the current release; stages 9–12 represent conceptual extensions and are not automated.

6 Discussion and Limitations

The system demonstrates how structured execution pipelines can support reproducible biomedical research workflows. The biomedical workflows are presented as reproducible computational demonstrations on synthetic and mock fixtures rather than as clinical validation studies.

6.1 Limitations

- The biomarker and causal-inference workflows use partially simulated data by default; real cohort or GWAS inputs are supported but not required.
- Limited real-world clinical validation is performed.
- No claims of clinical deployment or diagnostic use are made.

6.2 Reproducibility and Availability

The framework ensures reproducibility via fixed random seeds for all stochastic steps, continuous integration across Python 3.9–3.12, and a standardized output schema shared by all interface layers. Under identical inputs and a fixed seed, the framework produces an identical output structure across supported Python versions (validated in CI).

This paper describes a frozen system snapshot. The core framework is pinned at commit `2483d99` (branch `main`); the standardized interface package at commit `cc5911e` (branch `feat/skills-layer`); and this submission (paper source and bundled artifacts) at release tag `v1.0.0-arxiv-public` (branch `docs/arxiv-public`). All artifacts are archived at the cited commits and are verifiable by reviewers. A full reproducibility appendix, including the CI matrix and the reproducibility check, accompanies this submission.

6.3 Conclusion

We introduce a tool-first biomedical workflow framework that integrates reproducible execution, standardized interfaces, and consistent output formatting. The system is released with a CLI, SDK, API invocation contract, and a standardized external interface, all sharing a single reproducible workflow runner.

References

- [1] Eric W. Sayers, J. Beck, E. E. Bolton, D. Bourexis, J. R. Brister, K. Canese, D. C. Comeau, K. Funk, S. Kim, et al. Database resources of the national center for biotechnology information. *Nucleic Acids Research*, 49(D1):D10–D17, 2021.
- [2] Matthew E. Ritchie, Belinda Phipson, Di Wu, Yifang Hu, Charity W. Law, Wei Shi, and Gordon K. Smyth. limma powers differential expression analyses for rna-sequencing and microarray studies. *Nucleic Acids Research*, 43(7):e47, 2015.
- [3] Aravind Subramanian, Pablo Tamayo, Vamsi K. Mootha, Sayan Mukherjee, Benjamin L. Ebert, Michael A. Gillette, Amanda Paulovich, Scott L. Pomeroy, Todd R. Golub, Eric S. Lander, et al. Gene set enrichment analysis: a knowledge-based approach for interpreting genome-wide expression profiles. *Proceedings of the National Academy of Sciences*, 102(43):15545–15550, 2005.
- [4] Gibran Hemani, Jie Zheng, Benjamin Elsworth, Kaitlin H. Wade, Valeriia Haberland, Denis Baird, Charles Laurin, Stephen Burgess, Jack Bowden, et al. The mr-base platform supports systematic causal inference across the human phenome. *eLife*, 7:e34408, 2018.
- [5] Fernando Pires Hartwig, George Davey Smith, and Jack Bowden. Two-sample mendelian randomization: avoiding the crippling effects of population stratification. *International Journal of Epidemiology*, 46(6):1985–1998, 2017.

7 Supplementary Material

7.1 A. Continuous Integration Matrix

Table 1 summarizes the CI configuration that validates reproducibility across Python versions. Each cell denotes a workflow executed and asserted against the output contract.

7.2 B. Reproducibility Check

The causal workflow accepts `--seed` (default 42). The `determinism` CI job runs the causal workflow twice with `--seed 42` and asserts that the two runs produce an identical output-file structure (same file paths and table

Table 1: Continuous-integration matrix. All three workflows are executed and their output contract (success, report_path, tables, figures) is asserted on every Python version under **ubuntu-latest**.

Python	Literature	Biomarker	Causal
3.9	yes	yes	yes
3.10	yes	yes	yes
3.11	yes	yes	yes
3.12	yes	yes	yes

shapes). The literature workflow (**--use-mock**) and biomarker workflow (**--use-synthetic**) run on fixed local fixtures and require no network access.

7.3 C. Output Categorization Schema

Every extracted claim is assigned a five-level evidence grade using a rule-based scheme:

Table 2: Output-category definitions used by the categorization scheme.

Grade	Definition
A	Multiple independent high-quality studies, or direct experimental validation.
B	Single high-quality study, or strong consensus across sources.
C	Moderate evidence; plausible but with limited replication.
D	Weak or indirect evidence; hypothesis-level only.
E	Assertion unsupported by retrieved evidence; excluded from conclusions.

7.4 D. Twelve-Stage Execution Diagram

The workflow runner executes twelve discrete stages: (1) question formulation, (2) hypothesis generation, (3) literature retrieval, (4) study design, (5) data ingestion, (6) statistical analysis, (7) bioinformatics, (8) output aggregation, (9) writing, (10) internal review, (11) critique, (12) format-

ted output. Each stage emits an execution output and passes through a validation gate.

7.5 E. Standardized Interface Schema

The bundled skill package exposes a single entry point (`run`) with a model-agnostic tool schema. It returns the same output contract as the CLI and SDK (Section 6.2) and introduces no new computational capability. The manifest pins `release_tag = v1.0.0-arxiv-public` and `frozen_commit = cc5911e`.

7.6 F. Provenance and Commit Binding

This submission is a frozen snapshot:

- Core framework: commit `2483d99` (branch `main`).
- Standardized interface: commit `cc5911e` (branch `feat/skills-layer`).
- Paper source and bundled artifacts: release tag `v1.0.0-arxiv-public` (branch `docs/arxiv-public`).

All artifacts are archived at the cited commits and are verifiable by reviewers.